



Module 3 Day 2: How Sweet is the Terminal?

Overview

In today's class, you will expand your command-line skills by working in your terminal to complete tasks that require file searching and bulk operations. Throughout class, you will complete a series of activities involving the `man` and file preview commands.

Class Objectives

By the end of class, you will be able to do the following:

- Identify and explain the structure of a terminal command
 - Explain how options modify the default behavior of a terminal command
 - Use commands like `head`, `tail`, `more`, and `less` to preview files in the command line
 - Combine the above commands in sequence to accomplish relevant IT tasks
-

01. Recap

In the prior class, we learned to use several new commands, including the following:

- Commands for navigating a file directory:
 - `pwd` displays the current working directory.
 - `ls` lists the directories and files in the current directory.
 - `cd` navigates into a directory.
 - `cd ../` navigates out of a directory.
 - `clear` clears out the terminal history on the page.
- Commands for making and removing files and directories:
 - `mkdir` creates a new directory.
 - `rmdir` removes a directory.
 - `touch` creates an empty file.
 - `rm` removes a file.

- Commands for moving and copying files:
 - `cp` copies files.
 - `mv` moves files.

02. Preview Commands

So far, we have covered commands used to navigate the file system and to create, move, and copy files.

There are times that IT and security professionals need to preview the contents of a file. For example:

- A security administrator may want to preview the top 10 lines of an email file to figure out to whom the email was sent.
- A network administrator may want to preview the bottom of a log file to see the last timestamp captured in a file.

There are multiple preview commands that can assist with these tasks.

- To preview and scroll through a whole file, we can use the `more` or `less` commands.
 - `more` is used to view a file one page at a time; the spacebar is then used to move to the next page while viewing a file.
 - `less` is similar to `more`, except it is used to scroll up and down in a file.
- To preview a file by a certain number of lines, we can use the `head` or `tail` command.
 - `head` displays the top 10 lines of a file
 - `tail` displays the bottom 10 lines of a file
- Ten is the default number of lines displayed, but this number can be easily changed.
 - The syntax for `head` is `head -<number> <file>`.
 - For example, `head -50 logfile.txt` will display the top 50 lines in the `logfile.txt` file.
 - The same syntax is used for the `tail` command.

Preview commands demonstration Setup

We will walk through how to use the preview commands in the following scenario:

- In your role as security analyst at ACME Corp, you are asked to examine several potential evidence files.
- Your manager provides a directory called `evidence_directory` that contains four files (`File1`, `File2`, `File3`, and `File4`).
- Your manager explains that these files are log files pulled from a secret computer at ACME Corp. There should be access logs that show who logged in to this secret computer.
- You must use the preview commands to determine which of these files are actually access logs.
- Once you determine that a file is an access log, document the last timestamp in the file.

Preview commands demonstration

1. First, navigate into the `evidence_directory` directory.
 - o **Heads Up:** This is located in the `/03-instructor/day1/preview_demonstration/` folder.
 - o Run `cd evidence_directory`.
2. Your manager told you there are four evidence files in this directory.
 - o Confirm this is correct by running `ls`.
 - o The output should confirm the presence of four files: `File1`, `File2`, `File3`, and `File4`.
3. Preview the first file with the `more` command in order to determine whether it is an access log file.
 - o First, use the `more` command by running:
 - `more File1`
 - o Note that this file clearly does not look like an access log file.
 - o Use the spacebar to scroll down through the file, one screen at a time.
4. Preview the next file using the `less` command.
 - o Use the `less` command by running `less File2`.
 - o This file also clearly does not look like an access log file.
 - o Use the up and down arrow keys to preview the file.
 - o To exit this preview, you need to press the Q key on your keyboard.
5. Use the `head` command to display the top 10 lines of `File3`.
 - o Run the command `head File3`.
 - o This also does not appear to be an access log file, but it only shows the top 10 lines.
 - o View a larger set of lines (30) by running `head -30 File3`.
 - o Now, it displays the top 30 lines, but none of the records look like access logs.
6. Next, open up the last file with the `head` command again.
 - o Run the command `head File4`.
 - o This clearly looks like an access log file, as it has times, usernames, and login times.
7. Lastly, because we have found the access log file, we now need to preview the bottom of the file to view the last timestamp.
 - o Run the command `tail File4`.
 - o Note that the last timestamp of the file shows that Sally logged out at 12/23/19 4:33pm.

Summary:

- `more` is used to view a file one page at a time, using the spacebar to navigate to the next page.
- `less` is similar to `more`, but is used to scroll up and down a page.
- `head` is used to preview the top 10 lines of a file.
- `tail` is used to preview the bottom 10 lines of a file.
- Adding a `-<number>` after `head` or `tail` will change the number of lines displayed in the preview.

03. Activity: Oh Henry, What Did You Do (Lab)

You will continue in your role as security analyst at Candy Corp, investigating the insider employee potentially selling secret recipes to Sugar Corp. Your manager at Candy has identified two possible insiders who may be working with Sugar: Henry and Ruth.

Your manager pulled some files from Henry and Ruth's computers without their knowledge. Some of the files contain gibberish, but some may have useful data. Your manager has asked you to preview the files to determine which files have readable text data that can be analyzed later.

- [Activity File: Oh Henry, What Did You Do?](#)

04. Review: Oh Henry, What Did You Do (Lab)

The instructor will review the activity with the class.

05. Data Streams and the `cat` Command

IT and security professionals are often tasked with combining files.

- For example, a security professional may need to combine several of the same type of log files into a single log file.

Combining files allows us to conduct one single data analysis, rather than analyzing multiple files. We can use the `cat` command to do this.

- `cat` is short for "concatenate," a word meaning "to link together."
- `cat` combines the designated files and displays the results back to the user, but does not save the result beyond the command line.
 - For example, to combine two separate files, `1.txt` and `2.txt`, you would run `cat 1.txt 2.txt`.
 - This would display the combined results of `1.txt` and `2.txt`, but not save the results beyond the command line.
- In order to save the results of the `cat` command, you have to redirect the **data stream**.
 - The term **data stream** is used to describe channels of data as they are processed and moved through a system.
 - Data streams are used to move data from one process or program to another.
 - The two most commonly used data streams are `stdin` and `stdout`, pronounced "standard in" and "standard out."

- In the context of a command, input data is usually the `argument` given to the program., and output data is the information returned by the program when the command is successful.
- Input data is streamed using the data stream `stdin`.
- Output data is streamed using the data stream `stdout`.

This will become clearer as we practice using redirection with `stdin` and `stdout` when using the `cat` command.

- To save the results of `cat`, we have to use one of the following:
 - `>` to write or overwrite a file
 - `>>` to write or add to a file
- For example, if you wanted to combine two separate files, `1.txt` and `2.txt`, into a single file called `combined_1_and_2.txt`, you would run the following command:
 - `cat 1.txt 2.txt > combined_1_and_2.txt`
 - This command will create a new file called `combined_1_and_2.txt`. If the file name already exists, it will overwrite it with the combined data from `1.txt` and `2.txt`.
 - `cat` takes data directly from `stdin` and directs it to `stdout`.
 - `1.txt` and `2.txt` is sent to `stdin`.
 - `cat` takes `stdin`, concatenates the data, and sends it to `stdout`.
 - The results of combining `1.txt` and `2.txt` is `stdout`.
 - `stdout` is then redirected and written to a file called `combined_1_and_2.txt`.
- Or if you wanted to combine two separate files, `3.txt` and `4.txt`, into a single file, called `combined_3_and_4.txt`, you would run the following command:
 - `cat 3.txt 4.txt >> combined_3_and_4.txt`
 - This command will create a new file called `combined_3_and_4.txt`. If the file name already exists, it will add the combined data from `3.txt` and `4.txt` to the bottom of the file.

cat demonstration setup

We will use the following scenario to walk through how to use the `cat` command:

- You are a security analyst at ACME Corp and have been tasked with combining several evidence log files from a potential rogue employee.
- Your manager provides you with a directory called `Logfile_evidence` that contains four files (`LogFile1`, `LogFile2`, `LogFile3`, and `LogFile4`). Each file contains logs from a different day.
- Your manager explains that your forensic team will do an analysis of these logs, but the team's job will be easier if you can combine the log files into a single file.
- You must use the `cat` command to combine them into a single file called `rogue_employee_log_evidence`.

cat demonstration

1. Start by navigating into the `Logfile_evidence` directory.
 - o **Heads Up:** This is located in the `/03-instructor/day1/cat_demonstration` folder.
 - o Run `cd Logfile_evidence`.
2. Confirm there are four log files in this directory.
 - o Run `ls`.
 - o There are four files in this directory: `LogFile1`, `LogFile2`, `LogFile3`, `LogFile4`.
 - o Use the `more` command to preview the files and confirm that each log file is a log for a different day.
 - o For example, run `more LogFile1`.
3. Use the `cat` command to combine these four log files, but first do so without saving.
 - o Type the following command: `cat LogFile1 LogFile2 LogFile3 LogFile4`.
 - o Note that the syntax is simply `cat` followed by the files we want to combine, separated by spaces.
 - o Run the command to see the results of combining the four files displayed on the screen.
 - o Note that the files `LogFile1`, `LogFile2`, `LogFile3`, and `LogFile4` are sent to `stdin` and the combined four files displayed is the `stdout`.
4. You were tasked by your manager to combine all four files into a file called `rogue_employee_log_evidence`.
 - o Type the following command: `cat LogFile1 LogFile2 LogFile3 LogFile4 > rogue_employee_log_evidence`.
 - o Note that the syntax is the same as the previous command, except we are using the `>` command to redirect the results into a file called `rogue_employee_log_evidence`.
 - o Run the command and then run `ls` to display a file called `rogue_employee_log_evidence`.
5. Finally, preview the file called `rogue_employee_log_evidence` to confirm that the four files were successfully combined.
 - o Run `more rogue_employee_log_evidence` and push the spacebar to move a page down through the file and confirm the combination of the four log files.

Summary:

- `cat` is used to concatenate and combine multiple files.
 - Redirection commands, such as `>` or `>>`, are used to combine multiple files into a single file.
 - o `>` will write to a file, but will also overwrite a pre-existing file if it shares the same name.
 - o `>>` will write to a file, but will append the combination to the end of a pre-existing file if it shares the same name.
-

06. Activity: Internal Investigation—Find the Kit cat Burglar (Lab)

In this activity, you will continue to play the role of a security analyst at Candy Corp investigating the employees potentially selling secret recipes to Sugar Corp. Your manager believes that Henry and Ruth are sharing secret recipes, and has collected some additional files from both of their computers in order to help build a case against them.

Now, your manager wants you to analyze this additional data and prepare a combined evidence file for the local authorities. You must act fast, so that the secret recipes don't get into the hands of Sugar Corp.

- [Activity File: Internal Investigation: Find the Kit cat Burglar](#)
-

07. Review: Internal Investigation—Find the Kit cat Burglar

The instructor will review the activity with the class.

08. Command-line Structure

So far, we have covered many basic command-line commands used to complete common IT tasks.

All of these commands have to follow a specific structure to run successfully:

- The structure that we are already familiar with is:
`<command> <argument>`
- **Arguments** appear immediately after the command, if they are inputs for the command.
 - For example, we know the following about `touch myfile`:
 - The command is `touch`.
 - The argument is `myfile`.

IT professionals often need to run commands with more specific parameters than can be included in the command itself.

- For example, IT analysts often need to clean up server space. They want to list the files by size, so they can delete the largest files first. However, the default behavior of the `ls` command does not list the files by size.
- Fortunately, there is a third element we can add to the structure of a command: **options** can modify the default behavior of a command.

There are three primary methods for using options.

Method 1: Add an option to modify a command's default behavior

Example: `ls -S`

- The added option modifies the behavior of the command. Adding `-S` after the `ls` command changes the behavior of the `ls` command from simply listing the files to listing them by size, with the largest first.
- We know the following about the syntax of the above command:
 - `ls` is the command.
 - `-S` is the option.
- Options always start with a hyphen.
- Options, just like commands, are case sensitive, meaning it matters whether we use capital or lowercase letters.
 - A lowercase `-s` will provide a very different result from an uppercase `-S`.
 - The lowercase `-s` option prints the size of each file.
- Each command has its own set of options. An option used in one command may behave completely differently in another.
 - For example:
 - `ls -s` will print the size of each file in a directory.
 - `cat -s` will suppress repeated empty output lines.

Method 2: Add an option and an argument to modify the default behavior of a command on files or directories

Example: `cat -n logfile1.txt`

- We can use this command to display line numbers on a file called `logfile1.txt`.
- The default behavior of the `cat` command is to concatenate multiple files or to simply display the contents of a single file.
- To modify the behavior of the `cat` command for a file, we can run `cat -n logfile1.txt`.
 - Adding the option `-n` modifies the behavior of the `cat` command to display the line numbers preceding each line.
- We know the following about the syntax of the above command:
 - `cat` is the command.
 - `-n` is the option.
 - `logfile1.txt` is the argument for the `cat` command.

Method 3: Add options that require their own arguments, called parameters

Example: `head -n 4 logfile1.txt`

- **Parameters** provide additional details about how a command's default behavior should be modified.
 - We can use the above command to preview the first four lines of `logfile1.txt`.
 - The default behavior of `head` will display the top 10 lines of a file.
 - The `-n` option modifies the default behavior of `head` by changing the number of lines it displays.
 - The `-n` option requires a parameter to specify the number of lines `head` should display.
 - We know the following about the syntax of the above command:
 - `head` is the command.
 - `-n` is the option (and this specific option requires a parameter).
 - `4` is the parameter for the `-n` option.
 - `logfile1.txt` is the argument for the `head` command.
-

09. Welcome to Man Pages

We have just learned that commands have default behaviors, and options can modify this default behavior to perform additional tasks.

Each command has the following:

- Its own unique list of options
- Certain options that require parameters

So, how can IT and security professionals keep track of all these options for all these commands?

- The command line has a valuable resource known as the **manual** or **man pages**.
- Man pages are a form of documentation available on the terminal.
- Each command has an associated man page.
- Each man page contains the following:
 - The name of the command
 - A synopsis that includes the syntax of the command
 - A description of the command
 - The available options and option parameters for the command

We can easily display the man page for a command using `man <command>`.

- For example, `man ls` will display the man page of the `ls` command.

Man pages demonstration setup

Man pages are useful to IT and security professionals when they are learning new commands.

In the following scenario, we will demonstrate how man pages can help you learn a new command.

- You are a security analyst at ACME Corp, and your manager asks you to count the number of server logins on October 13, 2019, as they believe it is higher than usual.
- They tell you to use the command `wc` to count the number of logins on the server login file.
- You have not used the `wc` command before, and will need to use the man page to learn how to do so.

Man pages demonstration

The instructor will demonstrate the use of `man wc` to access the man pages for the `wc` command.

Summary:

- Man pages refer to documentation that exists in the terminal and provides details about command-line commands and their accompanying options and parameters.
 - `man <command>` is the syntax used to display the man page for a particular command.
 - Man pages can be used to learn about new commands and the options of those commands.
-

10. Activity: Learn New Commands (Lab)

In this activity, you will continue with the scenario in which you are a security analyst at Candy Corp. This time, Candy Corp has recently experienced a network attack on its websites and management needs your help determining which website was the main target of the attack.

Your manager provides you with log files for each of the websites. The log files contain the IP addresses that connected to each website on the day of the attack. You must count the IP addresses in each file in order to determine which Candy website was the main target of the attack.

- [Activity File: Learn New Commands](#)
-

11. Review: Learn New Commands (Lab)

The instructor will review the activity with the class.

12. Summary

Congratulations on your hard work and dedication throughout the lesson! Let's recap what we have learned about the following:

- **Data streams and the `cat` command:**
 - IT and security professionals use the `cat` command to concatenate multiple files for a single analysis.
 - Data streams (`stdin` and `stdout`) can be used to redirect output to save combined files.
- **Using `wc` for counting:**
 - The `wc` command is used to count lines, words, and characters in files, which is crucial for analyzing log files and other data sets.
- **Man pages:**
 - Man pages are used to find detailed documentation and options for command-line commands and learn about new commands and their options.

Continue practicing these commands and exploring additional command-line tools to further enhance your IT and security skills. These foundational skills are vital for effective data management and analysis in your future careers.